Advanced Applied Cryptography Project Assignment 2

Diogo Nunes Nº70502

Pedro Galego Nº72298

1. Introduction and Project Goal

This project implements a privacy-preserving employee database for an outsourced storage scenario. The main goal is to allow a client to store employee records on an untrusted server while keeping plaintext employee information and cryptographic keys on the client side only. The server stores encrypted attributes and performs selected searches, ordering operations, and homomorphic computations directly over encrypted data.

The implementation follows a simple client/server model. A bootstrap program reads the employee dataset, encrypts the required fields, stores the encrypted records in a MySQL table, and writes a client-side index containing the cryptographic material needed by later client operations. A query processor then loads the client index, issues SQL queries over encrypted columns, and decrypts or verifies the results locally on the client.

2. System Design Assumptions

2.1 Threat Model

- The server is honest-but-curious: it correctly stores records and executes SQL queries, but it may inspect the encrypted database and observe query patterns.
- The server does not possess secret keys, private keys, or the client-side index. All decryption is performed by the client.
- The client is trusted and is responsible for bootstrap, key management, query token/encrypted predicate generation, and result decryption.
- The database backend is MySQL. The database is treated as remote/untrusted.

2.2 Architecture

The system is divided into three main components: the client bootstrap, the encrypted database, and the client query processor.

Component	Role in the System
Client Bootstrap	Reads the CSV dataset, generates cryptographic keys, encrypts employee attributes, creates the encrypted_employees table, inserts encrypted records, and writes client_index.json.
Encrypted MySQL Database	Stores only encrypted/searchable columns such as deterministic AES fields, randomized AES fields, OPE values, Paillier salary ciphertexts, and RSA salary ciphertexts.
Client Query Processor	Loads client_index.json, encrypts query predicates, sends SQL queries over encrypted columns, receives encrypted results, and decrypts/prints the final output on the client side.

2.3 Database Encryption Choices

The encrypted table uses separate encrypted representations of the same plaintext attribute when different operations require different cryptographic properties. For example, salary is represented with OPE for ordering, Paillier for additive computations, and RSA for multiplicative conversion.

Attribute / Purpose	Stored Column(s)	Cryptographic Method	Reason
Employee ID exact search	employeeID_DET	Deterministic AES-CTR	Allows equality search by encrypting the search value on the client and matching ciphertexts on the server.
Full name retrieval and exact search	fullName_RND, fullName_DET	Randomized AES-CTR and deterministic AES-CTR	Randomized encryption protects the displayed field; deterministic encryption supports exact name lookup.
Age ordering and oldest employee	age_OPE	Order-Preserving Encryption mapping	Allows ORDER BY operations over encrypted age values.
Department search	departmentID_DET	Deterministic AES-CTR	Allows exact-match department queries.
Salary ordering/comparison/highest salary	salary_OPE	Order-Preserving Encryption mapping	Allows salary ordering and direct comparison of encrypted salary order values.
Salary sum and bonus calculation	salary_PAILLIER	Paillier additive homomorphic encryption	Allows encrypted payroll sums and scalar multiplication for 25% bonus computation.
Salary EUR-to-USD conversion	salary_RSA	Unpadded RSA multiplicative homomorphic encryption	Allows multiplicative conversion using an illustrative exchange-rate multiplier.
Bonus eligibility search	bonusEligibility_DET	Deterministic AES-CTR	Allows filtering bonus-eligible employees.

2.4 Bootstrap Process and Client-Side Index

1. Generate cryptographic material: AES deterministic key, AES randomized key, Paillier key pair, RSA key pair, and an OPE mapping object.
2. Create or reset the MySQL encrypted_employees table.
3. Read each employee record from the CSV dataset.
4. Encrypt each attribute using the representation needed for the supported operations.
5. Insert only encrypted values into the database.
6. Save client_index.json containing client-side keys and OPE mapping information. This file must remain private to the client and must not be stored on the server.

3. Supported Operations

The query processor demonstrates the required client operations by producing encrypted predicates on the client, sending SQL queries over encrypted database columns, and decrypting final results locally.

Operation	Encrypted Processing Strategy
Search by Employee Identification	Encrypt the requested employee ID deterministically and use WHERE employeeID_DET = ?.
Search by Full Name	Encrypt the requested full name deterministically and use WHERE fullName_DET = ?.
Retrieve employees ordered by salary	Use ORDER BY salary_OPE DESC. The client decrypts names and OPE salary values.
Search employees by Department	Encrypt the department ID deterministically and filter by departmentID_DET.
Find employee with highest salary	Use ORDER BY salary_OPE DESC LIMIT 1.
Compare two employees salaries	Retrieve both OPE salary values by deterministic full-name lookup and compare the encrypted order values.

Retrieve employees ordered by age	Use ORDER BY age_OPE ASC.
Convert salary to US Dollars	Retrieve RSA-encrypted salary and multiply it homomorphically by an encrypted illustrative conversion multiplier.
Department payroll sum	Filter employees by deterministic encrypted department and multiply Paillier ciphertexts to obtain an encrypted sum.
Find oldest employee	Use ORDER BY age_OPE DESC LIMIT 1 and decrypt the selected record on the client.
Compute 25% bonus for eligible employees	Filter deterministic encrypted bonus eligibility and use Paillier scalar multiplication by 25, then divide by 100 after client-side decryption.

4. Security Analysis

4.1 Confidentiality Properties

Plaintext employee records are not stored in the encrypted database table. The client performs encryption before insertion and decryption after retrieval. The database server therefore stores encrypted identifiers, names, departments, ages, salaries, and bonus eligibility values.

Randomized AES-CTR is used for the display copy of the full name, so repeated names do not necessarily produce the same ciphertext in that column. Paillier encryption is randomized and is suitable for additive homomorphic operations such as summing encrypted salaries. The OPE and deterministic AES columns intentionally trade some leakage for server-side search and ordering capability.

4.2 Leakage Discussion

Source of Leakage	What the Server Can Learn
Deterministic AES columns	Equality patterns and frequency of repeated values. For example, employees in the same department have the same encrypted department value.
OPE columns	The relative order of salaries and ages, and therefore which encrypted value is larger or smaller. This is necessary for ORDER BY and comparison operations but leaks ordering information.
Query predicates and result sizes	Access patterns, repeated searches, and how many records match a department, employee name, or bonus eligibility value.
Unpadded RSA salary column	Multiplicative homomorphism is available, but unpadded RSA is deterministic and not semantically secure. Equal salaries produce equal RSA ciphertexts, and this representation should be used only for the project demonstration.
Client index compromise	If client_index.json is leaked, the attacker obtains cryptographic keys and the OPE mapping, making database confidentiality ineffective.

4.3 Leakage Minimization Measures

- Keep all keys, private parameters, and OPE mappings in the client index only.
- Use randomized AES encryption for fields that only need to be retrieved and displayed, rather than searched.
- Use deterministic encryption only where exact-match search is required.
- Use Paillier encryption for additive operations so the server can compute sums without seeing individual salaries or totals.
- Use separate encrypted representations of salary so that each representation supports only the required function.
- Add HMAC tags and ECDSA signatures per row in a final hardening pass, then verify them client-side after retrieval.
- Restrict database privileges and avoid exposing client_index.json to the server, logs, or repository history.

5. Performance Evaluation

The performance evaluation was conducted by instrumenting the Java bootstrap code with System.nanoTime(). The bootstrap latency measures the time observed by the client to generate keys, encrypt all employee records, insert the encrypted database into MySQL, and save the client-side index.

5.1 Bootstrap Latency

The following table shows the bootstrap latency measured across ten executions of MainBootstrap. The measurements are reported in both milliseconds and seconds.

Run	Bootstrap latency (ms)	Bootstrap latency (s)
1	4217.047	4.217
2	2782.434	2.782
3	1828.430	1.828
4	1987.751	1.988
5	2146.592	2.147
6	1821.096	1.821
7	1857.316	1.857
8	2205.176	2.205
9	1920.990	1.921
10	1924.490	1.924
Average	**2269.132**	**2.269**

The average bootstrap latency was 2269.132 ms, or approximately 2.269 seconds. The first execution was the slowest at 4217.047 ms, which is consistent with warm-up effects such as JVM initialization, database connection setup, class loading, and cryptographic provider initialization.

5.2 Encryption Time per Algorithm

The encryption-time benchmark measures only the time spent inside the encryption calls used to create encrypted database attributes. The dataset used in this benchmark contains 20 employee records.

Algorithm	Encrypted attribute(s)	Operations	Total time (ms)	Average time (ms/op)
AES/CTR deterministic	Employee ID, full name deterministic copy, department ID, bonus eligibility	80	19.434	0.242928
AES/CTR randomized	Full name randomized display copy	20	2.512	0.125576
OPE	Age and salary order values	40	0.512	0.012801
Paillier 2048-bit	Salary for additive homomorphic operations	20	248.898	12.444901
RSA 2048-bit	Salary for multiplicative homomorphic conversion	20	1.645	0.082240

The results show that Paillier encryption dominates the encryption cost during bootstrap. Even though Paillier is applied only once per employee salary, its average cost was 12.444901 ms per encryption, much higher than the other algorithms.

AES/CTR operations were comparatively fast, while OPE was the fastest.

Overall, the benchmark confirms that the main bootstrap cost comes from key generation, database insertion, and Paillier salary encryption. The measured per-algorithm encryption times also explain why attributes encrypted with lightweight symmetric or order-preserving methods contribute much less to total bootstrap latency than the homomorphic salary representation.

6. Conclusion

This project demonstrates how a database can support useful operations while keeping employee information encrypted at rest on an untrusted server. The implementation uses deterministic encryption for exact-match search, OPE for ordering and comparison, Paillier encryption for additive computations, and RSA multiplicative homomorphism for the illustrative currency-conversion operation. The main security tradeoff is controlled leakage: exact-match, ordering, and access-pattern leakage are accepted so that the server can process the required operations without plaintext access.